Revue & Rétrospective — Sprint 3 : Backend & Dashboard

1. Revue du sprint — fonctionnalités terminées

- Les tables **hosts**, **alerts** et **traffic_stats** sont créées en SQLite (database.py) et la classe **AlertRepository** (enregistrer, lister avec filtres, changer le statut) est implémentée et couverte par 7 tests unitaires.
- Le dashboard (cartes de synthèse, graphique trafic/anomalies, tableau des alertes) et l'API Flask (statistiques, trafic, liste des alertes, mise à jour du statut) fonctionnent et sont connectés entre eux. Ils affichent le trafic réel du Mac capturé avec tcpdump, en plus des données de démonstration.
- Les filtres par statut et par sévérité ainsi que le bouton **Traiter / Rouvrir** sont opérationnels.
- **AbuseIPDB** est intégré : le module reputation.py interroge l'API pour les adresses publiques uniquement, garde les réponses 24 h en mémoire, respecte un budget quotidien et ne bloque jamais le dashboard en cas d'erreur (30 tests). Deux colonnes (reputation_score, country) ont été ajoutées à la table des alertes, avec migration automatique des anciennes bases, et une colonne « Réputation » apparaît dans le tableau (testé sur des alertes réelles).
- Le dashboard passe 44 tests automatisés (39 existants + 5 sur la migration et l'enrichissement).

Statut : toutes les tâches du Sprint 3 sont terminées, y compris la tâche Souhaitable (AbuseIPDB).

2. Rétrospective

Ce qui a bien fonctionné

- Le module AbuseIPDB a été conçu pour être testé sans réseau (fonction d'appel injectable) : les erreurs 401, 422, 429 et les coupures réseau sont toutes couvertes.
- La migration des bases existantes se fait sans perte : les données de démonstration (demo.db) et les données réelles (reel.db) sont restées séparées et utilisables.
- Le fonctionnement a été vérifié visuellement sur de vraies alertes du Mac, pas seulement par des tests.

Difficultés rencontrées

- Une partie du dashboard (interface, API, moteur de détection basé sur tcpdump) a été développée dans une conversation distincte. Cela a créé plusieurs copies du dossier (v4, v5) et un décalage avec mes documents de conception (Scapy, AlertRepository, noms de tables). Corrigé en identifiant la version à jour par comparaison des dates et des fichiers, et en gardant une seule copie de référence.
- Le port 5000 est occupé par AirPlay sur macOS (erreur 403) : le dashboard a été relancé sur le port 5001.
- Lancer pytest à la racine échoue quand plusieurs copies du projet contiennent des fichiers de test du même nom. Corrigé en lançant les tests dossier par dossier.
- Le Python installé sur le Mac ne trouvait pas les certificats de sécurité (erreur SSL) pour contacter AbuseIPDB. Corrigé avec le paquet certifi.
- L'adresse IPv6 du Mac change régulièrement : elle aurait pu être envoyée à AbuseIPDB comme une adresse externe. Corrigé en ignorant toutes les adresses du réseau local.

Changements pour le prochain sprint

- Au Sprint 4, brancher le vrai pipeline (capture Scapy → AnomalyDetector → base du dashboard) et convertir les scores d'Isolation Forest (négatifs) vers l'échelle 0–1 attendue par le dashboard.
- Choisir une seule source de vérité pour la base de données (celle du dashboard ou AlertRepository) et mettre à jour les documents de spécification (API, composants, tables) pour qu'ils décrivent ce qui est réellement implémenté.
- Ne conserver qu'un seul dossier de dashboard, ne jamais écrire de clé d'API dans le code ou dans un message, et ne lancer qu'une seule capture à la fois.